In [ ]:
```
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from statsmodels.formula.api import ols
```

In [ ]:
```
from google.colab import files
files.upload()
```

In [ ]:
```
import pandas as pd

try:
    df = pd.read_csv('/content/Global_Superstore2 .csv', encoding='latin1')
except UnicodeDecodeError:
    df = pd.read_csv('/content/Global_Superstore2 .csv', encoding='ISO-8859-1')

df.head()
data = df.copy()
data
```

Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	
0	32298	CA-2012-124891	31-07-2012	31-07-2012	Same Day	RH-19495	Rick Hansen	Consumer	New
1	26341	IN-2013-77878	05-02-2013	07-02-2013	Second Class	JR-16210	Justin Ritter	Corporate	Wollon
2	25330	IN-2013-71249	17-10-2013	18-10-2013	First Class	CR-12730	Craig Reiter	Consumer	Bris
3	13524	ES-2013-1579342	28-01-2013	30-01-2013	First Class	KM-16375	Katherine Murray	Home Office	
4	47221	SG-2013-4320	05-11-2013	06-11-2013	Same Day	RH-9495	Rick Hansen	Consumer	
...	...	...	...	...	...	...	...	...	
51285	29002	IN-2014-62366	19-06-2014	19-06-2014	Same Day	KE-16420	Katrina Edelman	Corporate	
51286	35398	US-2014-102288	20-06-2014	24-06-2014	Standard Class	ZC-21910	Zuschuss Carroll	Consumer	Ho
51287	40470	US-2013-155768	02-12-2013	02-12-2013	Same Day	LB-16795	Laurel Beltran	Home Office	O
51288	9596	MX-2012-140767	18-02-2012	22-02-2012	Standard Class	RB-19795	Ross Baird	Home Office	Val
51289	6147	MX-2012-134460	22-05-2012	26-05-2012	Second Class	MC-18100	Mick Crebagga	Consumer	Tip

51290 rows × 24 columns

In [ ]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51290 entries, 0 to 51289
Data columns (total 24 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Row ID          51290 non-null  int64
 1   Order ID        51290 non-null  object
 2   Order Date      51290 non-null  object
 3   Ship Date       51290 non-null  object
 4   Ship Mode       51290 non-null  object
 5   Customer ID     51290 non-null  object
 6   Customer Name   51290 non-null  object
 7   Segment         51290 non-null  object
 8   City            51290 non-null  object
 9   State           51290 non-null  object
 10  Country         51290 non-null  object
 11  Postal Code     9994 non-null   float64
 12  Market          51290 non-null  object
 13  Region          51290 non-null  object
 14  Product ID      51290 non-null  object
 15  Category        51290 non-null  object
 16  Sub-Category    51290 non-null  object
 17  Product Name    51290 non-null  object
 18  Sales           51290 non-null  float64
 19  Quantity        51290 non-null  int64
 20  Discount        51290 non-null  float64
 21  Profit          51290 non-null  float64
 22  Shipping Cost   51290 non-null  float64
 23  Order Priority  51290 non-null  object
dtypes: float64(5), int64(2), object(17)
memory usage: 9.4+ MB
```

In [ ]:
```
#T-Test: Compare numerical variables (like Sales, Profit) across two groups (e.g
from scipy.stats import ttest_ind

# Group data by Ship Mode
standard_class_sales = data[data['Ship Mode'] == 'Standard Class']['Sales']
first_class_sales = data[data['Ship Mode'] == 'First Class']['Sales']

# Perform the t-test
t_stat, p_val = ttest_ind(standard_class_sales, first_class_sales, equal_var=Fal

print(f"T-statistic: {t_stat}")
print(f"P-value: {p_val}")

# Interpretation
if p_val < 0.05:
    print("There is a significant difference in average sales between Standard C
else:
    print("No significant difference in average sales between Standard Class and
```

```
T-statistic: 0.36742177515410834
P-value: 0.7133111336352411
No significant difference in average sales between Standard Class and First Clas
s.
```

In [ ]:
```
#Chi-Square Test: Check if there's an association between two categorical variab
from scipy.stats import chi2_contingency
```

```python
# Create a contingency table for Segment and Ship Mode
contingency_table = pd.crosstab(data['Segment'], data['Ship Mode'])

# Perform chi-square test
chi2_stat, p_val, dof, expected = chi2_contingency(contingency_table)

print(f"Chi-Square Statistic: {chi2_stat}")
print(f"P-value: {p_val}")

# Interpretation
if p_val < 0.05:
    print("There is a significant association between Segment and Ship Mode.")
else:
    print("Segment and Ship Mode are independent.")
```

```
Chi-Square Statistic: 18.274424263215177
P-value: 0.005581770901920851
There is a significant association between Segment and Ship Mode.
```

In [ ]:
```python
contingency_table
```

Out[ ]:

Ship Mode Segment	First Class	Same Day	Second Class	Standard Class
Consumer	3918	1467	5257	15876
Corporate	2245	728	3198	9258
Home Office	1342	506	1854	5641

In [ ]:
```python
data.head(4)
```

Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City
0	32298	CA-2012-124891	31-07-2012	31-07-2012	Same Day	RH-19495	Rick Hansen	Consumer	New York City
1	26341	IN-2013-77878	05-02-2013	07-02-2013	Second Class	JR-16210	Justin Ritter	Corporate	Wollongong
2	25330	IN-2013-71249	17-10-2013	18-10-2013	First Class	CR-12730	Craig Reiter	Consumer	Brisbane
3	13524	ES-2013-1579342	28-01-2013	30-01-2013	First Class	KM-16375	Katherine Murray	Home Office	Berlin

4 rows × 24 columns

In [ ]:

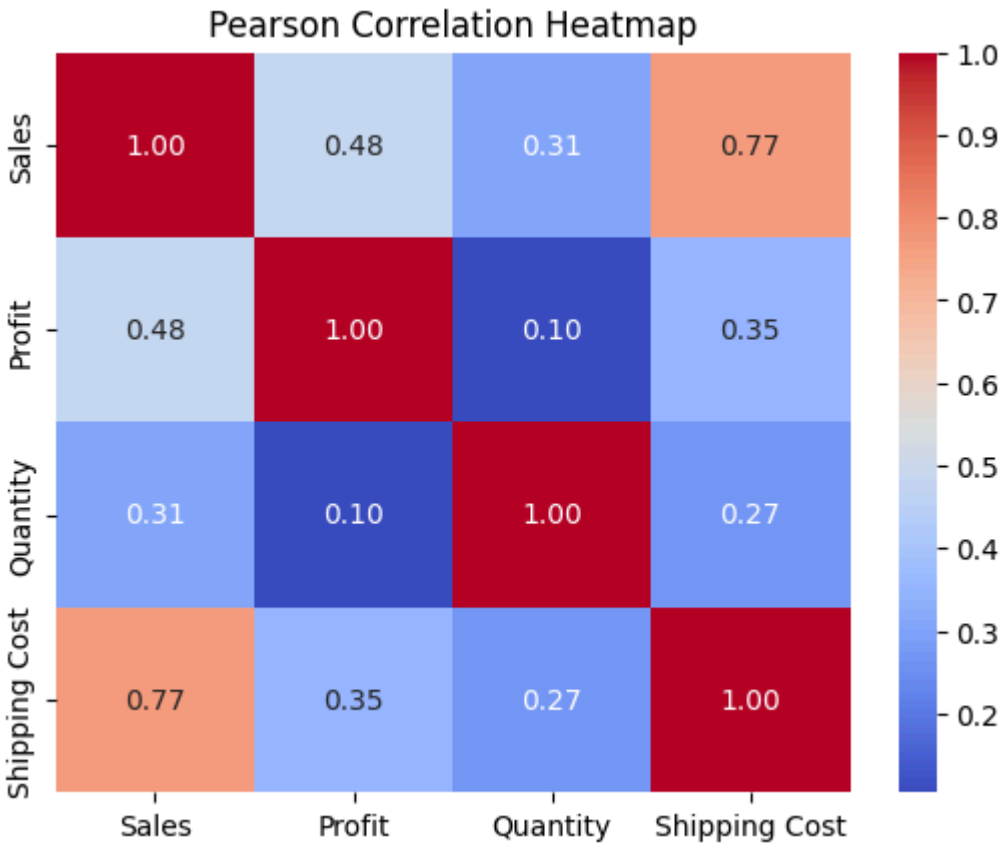
```
#Coreleation Analysis

# Pearson Correlation
pearson_corr = data[['Sales', 'Profit', 'Quantity', 'Shipping Cost']].corr(metho
print('Pearson Correlation:\n', pearson_corr)

# Spearman Correlation
spearman_corr = data[['Sales', 'Profit', 'Quantity', 'Shipping Cost']].corr(meth
print('Spearman Correlation:\n', spearman_corr)

# Heatmap
sns.heatmap(pearson_corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Pearson Correlation Heatmap')
plt.show()
```

```
Pearson Correlation:
                   Sales    Profit  Quantity  Shipping Cost
Sales          1.000000  0.484918  0.313577       0.768073
Profit         0.484918  1.000000  0.104365       0.354441
Quantity       0.313577  0.104365  1.000000       0.272649
Shipping Cost  0.768073  0.354441  0.272649       1.000000
Spearman Correlation:
                   Sales    Profit  Quantity  Shipping Cost
Sales          1.000000  0.490195  0.416397       0.913261
Profit         0.490195  1.000000  0.201442       0.449408
Quantity       0.416397  0.201442  1.000000       0.378736
Shipping Cost  0.913261  0.449408  0.378736       1.000000
```



In [ ]:
```
#For Annova testing columns choosen are profit and region
new_data = data[['Profit', 'Region']]
new_data
```

Out[ ]:

	Profit	Region
0	762.1845	East
1	-288.7650	Oceania
2	919.9710	Oceania
3	-96.5400	Central
4	311.5200	Africa
...	...	...
51285	4.5000	North Asia
51286	-1.1100	Central
51287	11.2308	West
51288	2.4000	South
51289	1.8000	Central

51290 rows × 2 columns

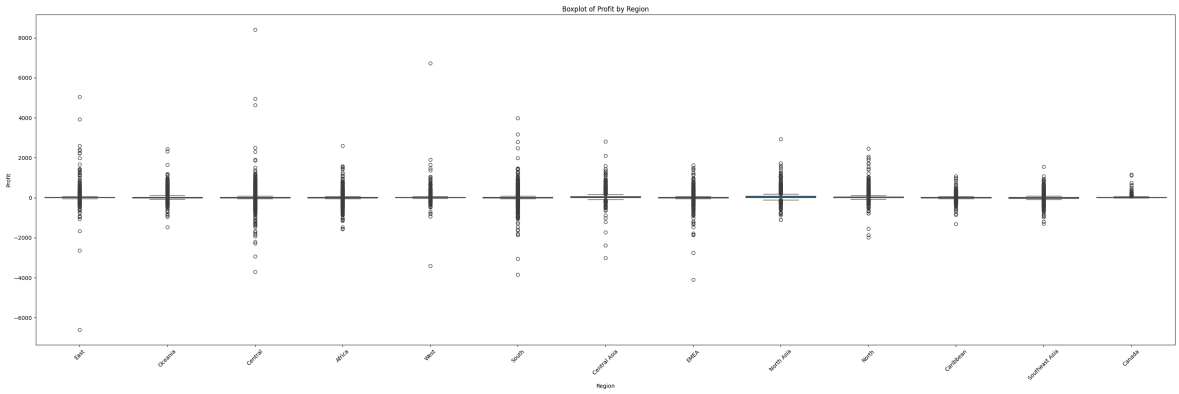
In [ ]:
```
data['Region'].nunique()
```

Out[ ]: 13

In [ ]:
```
plt.figure(figsize=(30, 10))  # Adjust width and height for elongation
sns.boxplot(x='Region', y='Profit', data=new_data)

# Rotate x-axis labels
plt.xticks(rotation=45)

plt.title('Boxplot of Profit by Region')
plt.tight_layout()  # Adjusts layout to prevent clipping of labels
plt.show()
```



In [ ]:
```
np.random.seed(0)  # For reproducibility
random_groups = np.random.choice([f'g{i+1}' for i in range(13)], size=len(data))
new_data['Random Region Group'] = random_groups
```

```
/tmp/ipykernel_404/1205691454.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stabl
e/user_guide/indexing.html#returning-a-view-versus-a-copy
  new_data['Random Region Group'] = random_groups
```

In [ ]:
```
new_data
```

Out[ ]:

	Profit	Region	Random Region Group
0	762.1845	East	g13
1	-288.7650	Oceania	g6
2	919.9710	Oceania	g1
3	-96.5400	Central	g4
4	311.5200	Africa	g12
...	...	...	...
51285	4.5000	North Asia	g6
51286	-1.1100	Central	g7
51287	11.2308	West	g7
51288	2.4000	South	g12
51289	1.8000	Central	g4

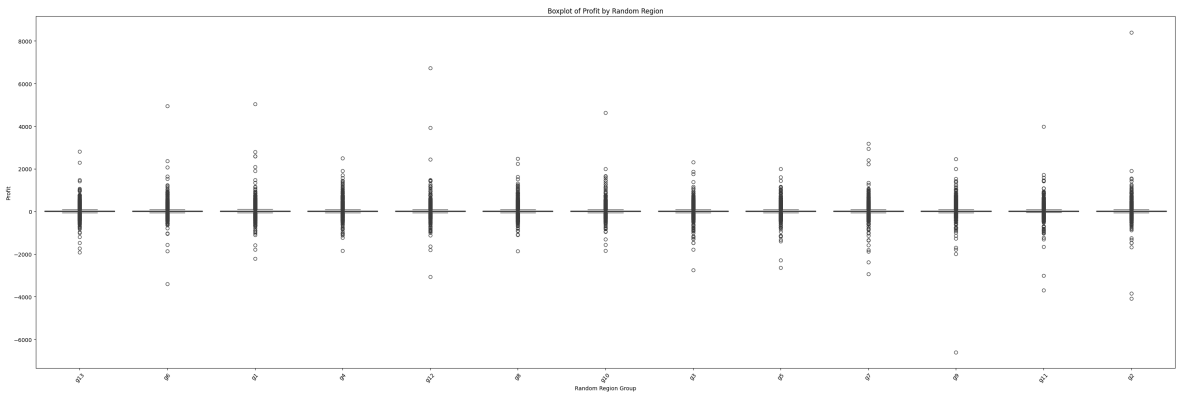
51290 rows × 3 columns

In [ ]:

```
plt.figure(figsize=(30, 10))  # Adjust width and height for elongation
sns.boxplot(x='Random Region Group', y='Profit', data=new_data)

# Rotate x-axis labels
plt.xticks(rotation=45)

plt.title('Boxplot of Profit by Random Region')
plt.tight_layout()  # Adjusts layout to prevent clipping of labels
plt.show()
```



In [ ]:

```
new_data['Region'].unique()
```

Out[ ]:

```
array(['East', 'Oceania', 'Central', 'Africa', 'West', 'South',
       'Central Asia', 'EMEA', 'North Asia', 'North', 'Caribbean',
       'Southeast Asia', 'Canada'], dtype=object)
```

In [ ]:

```
new_data['Random Region Group'].unique()
```

Out[ ]:

```
array(['g13', 'g6', 'g1', 'g4', 'g12', 'g8', 'g10', 'g3', 'g5', 'g7',
       'g9', 'g11', 'g2'], dtype=object)
```

In [ ]:
```
# Define profit data for each region
profit_east = new_data[new_data['Region'] == 'East']['Profit']
profit_oceania = new_data[new_data['Region'] == 'Oceania']['Profit']
profit_central = new_data[new_data['Region'] == 'Central']['Profit']
profit_africa = new_data[new_data['Region'] == 'Africa']['Profit']
profit_west = new_data[new_data['Region'] == 'West']['Profit']
profit_south = new_data[new_data['Region'] == 'South']['Profit']
profit_central_asia = new_data[new_data['Region'] == 'Central Asia']['Profit']
profit_emea = new_data[new_data['Region'] == 'EMEA']['Profit']
profit_north_asia = new_data[new_data['Region'] == 'North Asia']['Profit']
profit_north = new_data[new_data['Region'] == 'North']['Profit']
profit_caribbean = new_data[new_data['Region'] == 'Caribbean']['Profit']
profit_southeast_asia = new_data[new_data['Region'] == 'Southeast Asia']['Profit
profit_canada = new_data[new_data['Region'] == 'Canada']['Profit']
```

In [ ]:
```
# Perform One-Way ANOVA for Profit vs Region
import pandas as pd
from scipy.stats import f_oneway
f_stats, p_value = f_oneway(
    profit_east, profit_oceania, profit_central, profit_africa, profit_west,
    profit_south, profit_central_asia, profit_emea, profit_north_asia, profit_no
    profit_caribbean, profit_southeast_asia, profit_canada
)
print("One-Way ANOVA for Profit vs Region:")
print("Test statistic:", f_stats)
print("P-value:", p_value)

if p_value < 0.05:
    print("Reject H0: At least one region has a different mean profit.")
else:
    print("Fail to reject H0: All regions have the same mean profit.")
```

```
One-Way ANOVA for Profit vs Region:
Test statistic: 34.26121568235656
P-value: 3.617471983600004e-80
Reject H0: At least one region has a different mean profit.
```

In [ ]:
```
profit_g1 = new_data[new_data['Random Region Group'] == 'g1']['Profit']
profit_g2 = new_data[new_data['Random Region Group'] == 'g2']['Profit']
profit_g3 = new_data[new_data['Random Region Group'] == 'g3']['Profit']
profit_g4 = new_data[new_data['Random Region Group'] == 'g4']['Profit']
profit_g5 = new_data[new_data['Random Region Group'] == 'g5']['Profit']
profit_g6 = new_data[new_data['Random Region Group'] == 'g6']['Profit']
profit_g7 = new_data[new_data['Random Region Group'] == 'g7']['Profit']
profit_g8 = new_data[new_data['Random Region Group'] == 'g8']['Profit']
profit_g9 = new_data[new_data['Random Region Group'] == 'g9']['Profit']
profit_g10 = new_data[new_data['Random Region Group'] == 'g10']['Profit']
profit_g11 = new_data[new_data['Random Region Group'] == 'g11']['Profit']
profit_g12 = new_data[new_data['Random Region Group'] == 'g12']['Profit']
profit_g13 = new_data[new_data['Random Region Group'] == 'g13']['Profit']
```

In [ ]:
```
# Perform One-Way ANOVA for Profit vs Random Region Group
f_stats, p_value = f_oneway(
    profit_g1, profit_g2, profit_g3, profit_g4, profit_g5, profit_g6,
    profit_g7, profit_g8, profit_g9, profit_g10, profit_g11, profit_g12, profit_
)

print("\nOne-Way ANOVA for Profit vs Random Region Group:")
print("Test statistic:", f_stats)
```

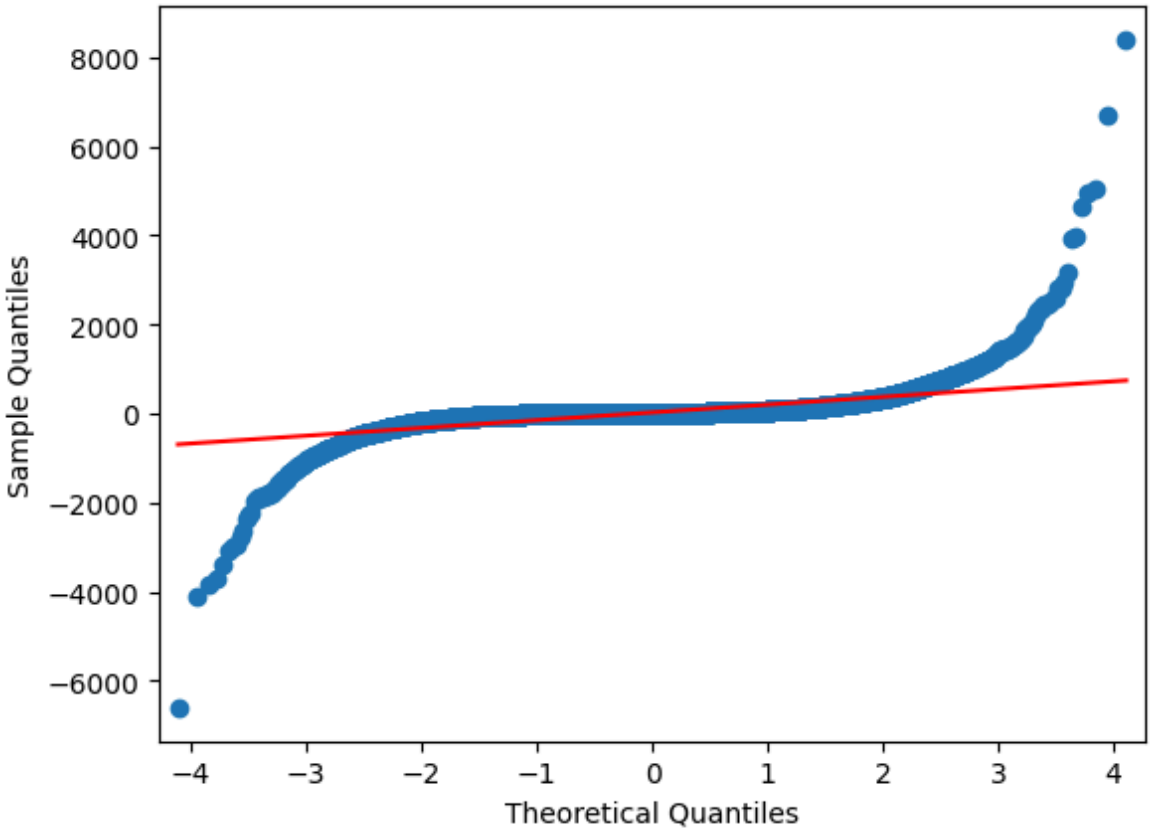
```
print("P-value:", p_value)

if p_value < 0.05:
    print("Reject H0: At least one random group has a different mean profit.")
else:
    print("Fail to reject H0: All random groups have the same mean profit.")
```

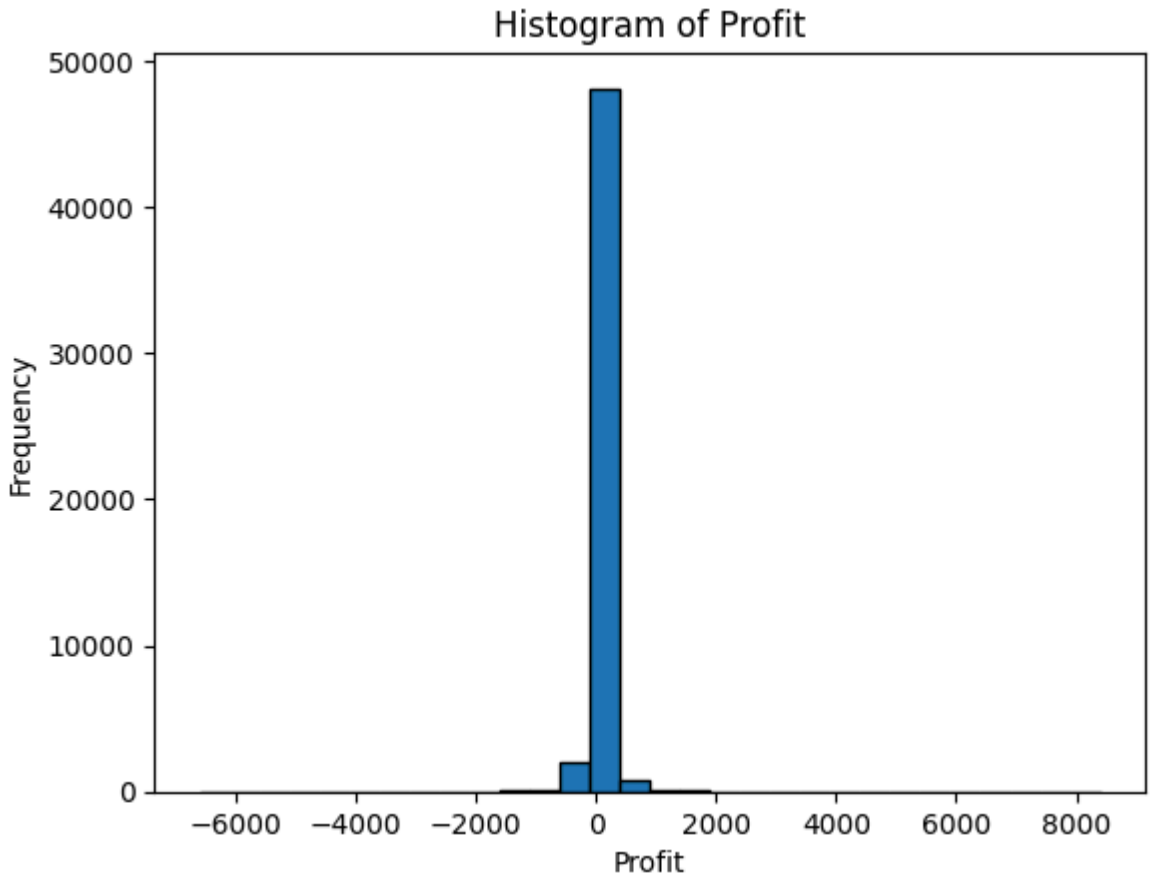
```
One-Way ANOVA for Profit vs Random Region Group:
Test statistic: 1.669498877166979
P-value: 0.0664825416601277
Fail to reject H0: All random groups have the same mean profit.
```

In [ ]:
```
import matplotlib.pyplot as plt
from statsmodels.graphics.gofplots import qqplot
```

In [ ]:
```
normality_check = new_data["Profit"]
qqplot(normality_check, line="s")
plt.show()
```



In [ ]:
```
#Normality check 2
# Plotting the histogram for Profit
plt.hist(new_data['Profit'], bins=30, edgecolor='black')
plt.title('Histogram of Profit')
plt.xlabel('Profit')
plt.ylabel('Frequency')
plt.show()
```



In [ ]:
```
# Perform Shapiro-Wilk test for normality
stat, p_value = stats.shapiro(new_data['Profit'])
print(f'Shapiro-Wilk Test: Stat={stat}, p-value={p_value}')

if p_value > 0.05:
    print("Data appears to be normally distributed (fail to reject H0).")
else:
    print("Data is not normally distributed (reject H0).")
```

```
Shapiro-Wilk Test: Stat=0.4478382879089836, p-value=1.4321003981764806e-145
Data is not normally distributed (reject H0).
```

```
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py:579: User
Warning: scipy.stats.shapiro: For N > 5000, computed p-value may not be accurate.
Current N is 51290.
  res = hypotest_fun_out(*samples, **kwds)
```

In [ ]:
```
# Perform Levene's test for equal variances across regions
stat, p_value = stats.levene(*[new_data[new_data['Region'] == region]['Profit']
print(f'Levene Test: Stat={stat}, p-value={p_value}')

if p_value > 0.05:
    print("Variances are equal across groups (fail to reject H0).")
else:
    print("Variances are not equal across groups (reject H0).")
```

```
Levene Test: Stat=14.297123030684464, p-value=2.592934018694379e-30
Variances are not equal across groups (reject H0).
```

In [ ]:
```
from scipy.stats import kruskal
stat, p_value = kruskal(
    profit_g1, profit_g2, profit_g3, profit_g4, profit_g5, profit_g6,
    profit_g7, profit_g8, profit_g9, profit_g10, profit_g11, profit_g12, profit_
)
```

```
print("\nKruskal-Wallis Test for Profit vs Random Region Group:")
print("Test statistic:", stat)
print("P-value:", p_value)

if p_value < 0.05:
    print("Reject H0: At least one random group has a different distribution of
else:
    print("Fail to reject H0: All random groups have the same distribution of pr
```

```
Kruskal-Wallis Test for Profit vs Random Region Group:
Test statistic: 13.761405113009443
P-value: 0.31620337263796416
Fail to reject H0: All random groups have the same distribution of profit.
```

In [ ]:

```
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import spearmanr

corr, p_value = spearmanr(new_data['Profit'], new_data['Region'])

print("Spearman's Correlation Coefficient:", corr)
print("P-value:", p_value)

if p_value < 0.05:
    print("The correlation is statistically significant.")
else:
    print("The correlation is not statistically significant.")
```

```
Spearman's Correlation Coefficient: -0.0072194539422763765
P-value: 0.10205061251738644
The correlation is not statistically significant.
```

In [ ]:

```
data.head()
```

Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City
0	32298	CA-2012-124891	31-07-2012	31-07-2012	Same Day	RH-19495	Rick Hansen	Consumer	New York City
1	26341	IN-2013-77878	05-02-2013	07-02-2013	Second Class	JR-16210	Justin Ritter	Corporate	Wollongong
2	25330	IN-2013-71249	17-10-2013	18-10-2013	First Class	CR-12730	Craig Reiter	Consumer	Brisbane
3	13524	ES-2013-1579342	28-01-2013	30-01-2013	First Class	KM-16375	Katherine Murray	Home Office	Berlin
4	47221	SG-2013-4320	05-11-2013	06-11-2013	Same Day	RH-9495	Rick Hansen	Consumer	Dakar

5 rows × 24 columns

In [ ]:
```
data.describe(include = ['object'])
```

Out[ ]:

	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City	
count	51290	51290	51290	51290	51290	51290	51290	51290	5
unique	25035	1430	1464	4	1590	795	3	3636	
top	CA-2014-100111	18-06-2014	22-11-2014	Standard Class	PO-18850	Muhammed Yedwab	Consumer	New York City	Calif
freq	14	135	130	30775	97	108	26518	915	

In [ ]:
```
data.drop('Random Region Group', axis=1, inplace=True, errors='ignore')
```

In [ ]:
```
numeric_data = data.select_dtypes(include=[float, int])
```

In [ ]:
```
kurtosis_values = numeric_data.kurt()
print(kurtosis_values)
```

```
Row ID            -1.200000
Postal Code       -1.493020
Sales            176.731200
Quantity           2.275889
Discount           0.716682
Profit           291.411090
Shipping Cost     50.020158
dtype: float64
```

In [ ]:
```
#as previously it was clear that the dataset has numerous outliers in it.
#Outliers detection

num_new_data = data.select_dtypes(include = np.number)
num_new_data
```

Out[ ]:

	Row ID	Postal Code	Sales	Quantity	Discount	Profit	Shipping Cost
0	32298	10024.0	2309.650	7	0.0	762.1845	933.57
1	26341	NaN	3709.395	9	0.1	-288.7650	923.63
2	25330	NaN	5175.171	9	0.1	919.9710	915.49
3	13524	NaN	2892.510	5	0.1	-96.5400	910.16
4	47221	NaN	2832.960	8	0.0	311.5200	903.04
...	...	...	...	...	...	...	...
51285	29002	NaN	65.100	5	0.0	4.5000	0.01
51286	35398	77095.0	0.444	1	0.8	-1.1100	0.01
51287	40470	93030.0	22.920	3	0.0	11.2308	0.01
51288	9596	NaN	13.440	2	0.0	2.4000	0.00
51289	6147	NaN	61.380	3	0.0	1.8000	0.00

51290 rows × 7 columns

In [ ]:
```
num_new_data
```

Out[ ]:

	Row ID	Postal Code	Sales	Quantity	Discount	Profit	Shipping Cost
0	32298	10024.0	2309.650	7	0.0	762.1845	933.57
1	26341	NaN	3709.395	9	0.1	-288.7650	923.63
2	25330	NaN	5175.171	9	0.1	919.9710	915.49
3	13524	NaN	2892.510	5	0.1	-96.5400	910.16
4	47221	NaN	2832.960	8	0.0	311.5200	903.04
...	...	...	...	...	...	...	...
51285	29002	NaN	65.100	5	0.0	4.5000	0.01
51286	35398	77095.0	0.444	1	0.8	-1.1100	0.01
51287	40470	93030.0	22.920	3	0.0	11.2308	0.01
51288	9596	NaN	13.440	2	0.0	2.4000	0.00
51289	6147	NaN	61.380	3	0.0	1.8000	0.00

51290 rows × 7 columns

In [ ]:
```
num_new_data.drop('Row ID', axis = 1, inplace = True)
```

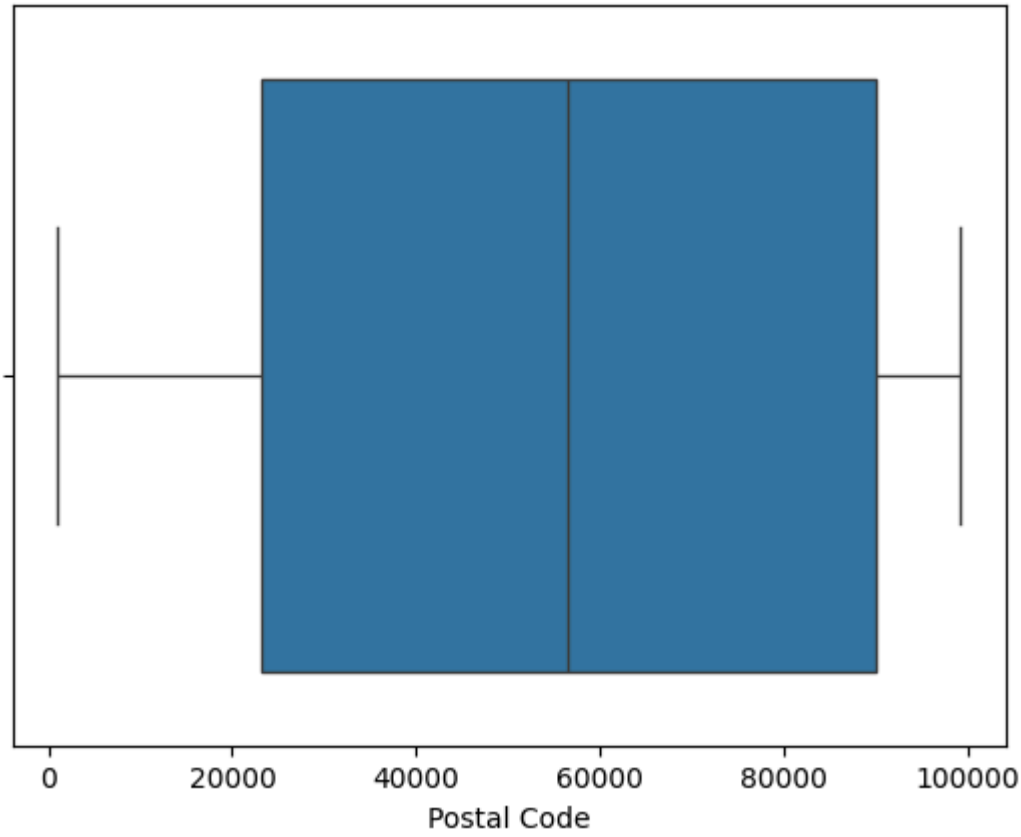
In [ ]:
```
num_new_data
```

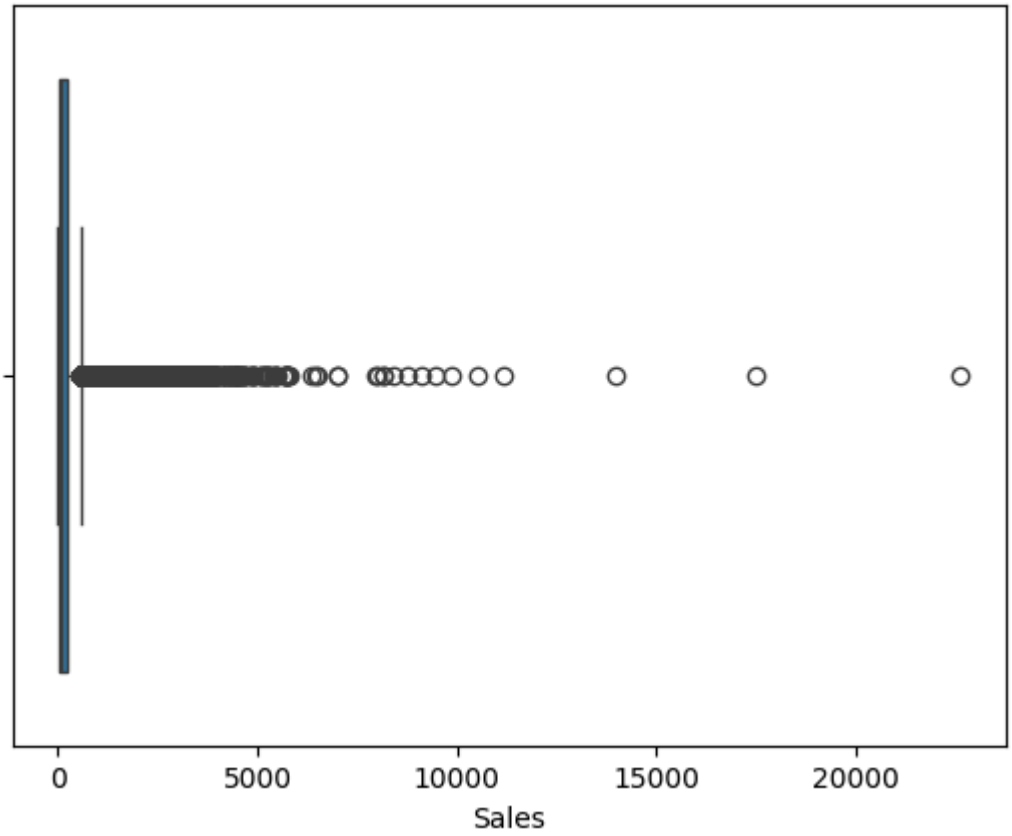
Out[ ]:

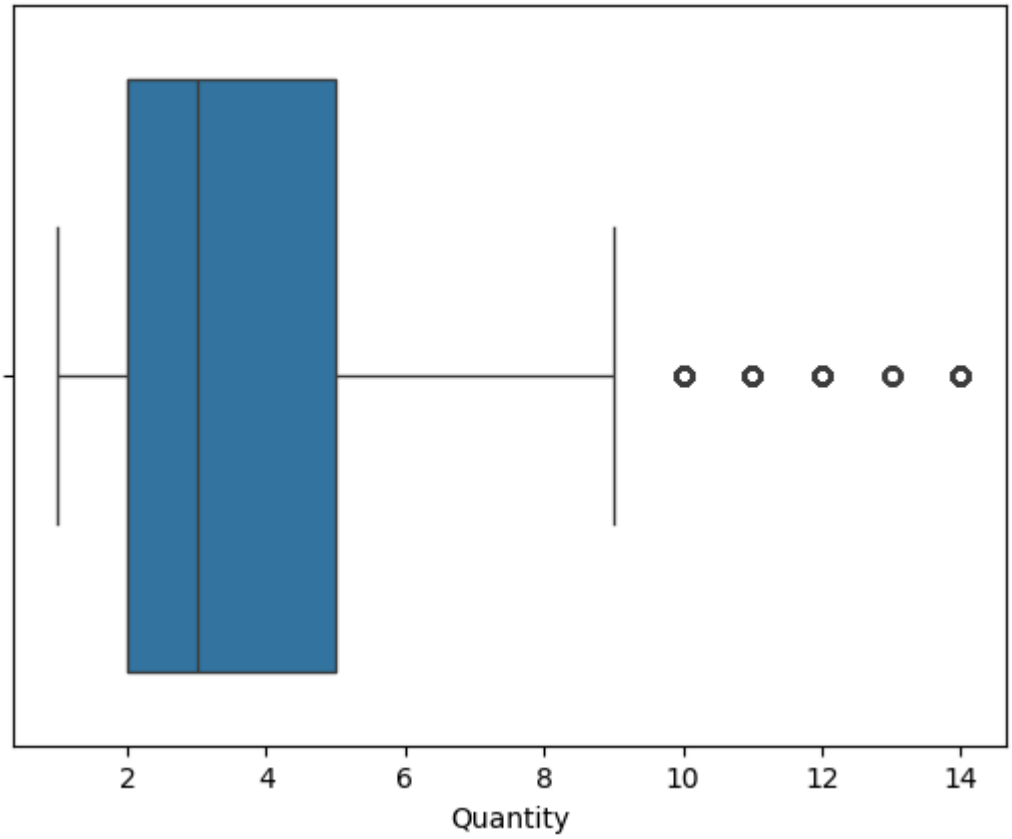
	Postal Code	Sales	Quantity	Discount	Profit	Shipping Cost
0	10024.0	2309.650	7	0.0	762.1845	933.57
1	NaN	3709.395	9	0.1	-288.7650	923.63
2	NaN	5175.171	9	0.1	919.9710	915.49
3	NaN	2892.510	5	0.1	-96.5400	910.16
4	NaN	2832.960	8	0.0	311.5200	903.04
...	...	...	...	...	...	...
51285	NaN	65.100	5	0.0	4.5000	0.01
51286	77095.0	0.444	1	0.8	-1.1100	0.01
51287	93030.0	22.920	3	0.0	11.2308	0.01
51288	NaN	13.440	2	0.0	2.4000	0.00
51289	NaN	61.380	3	0.0	1.8000	0.00

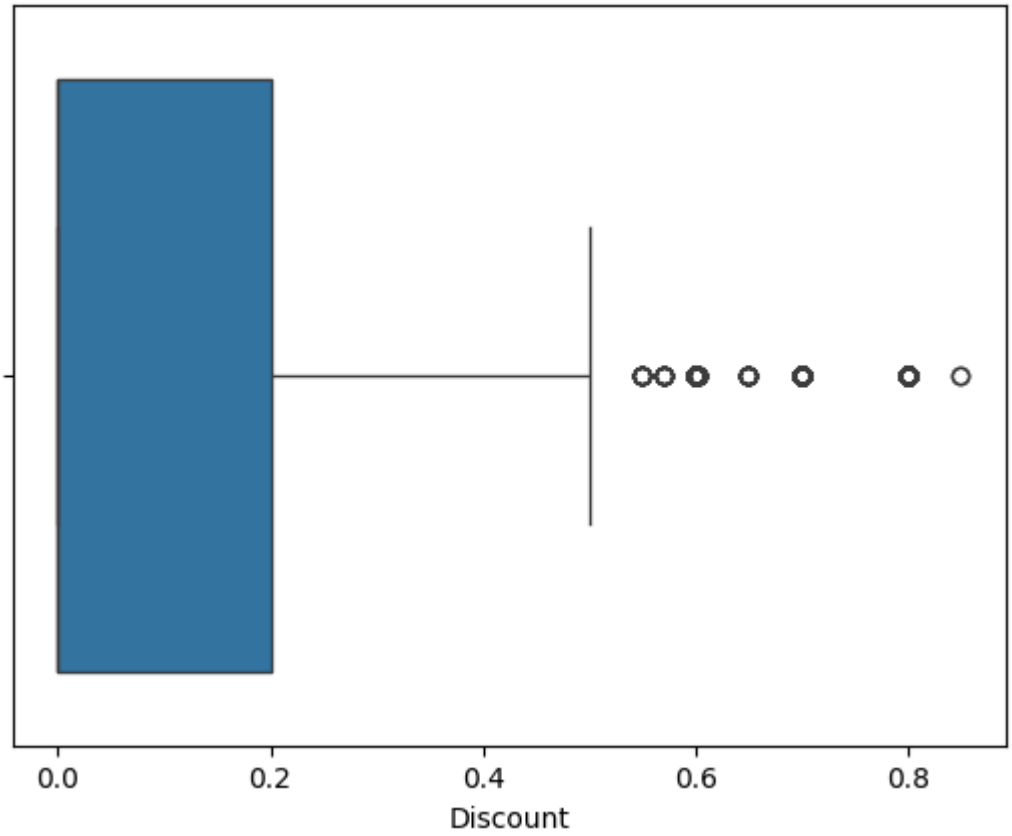
51290 rows × 6 columns

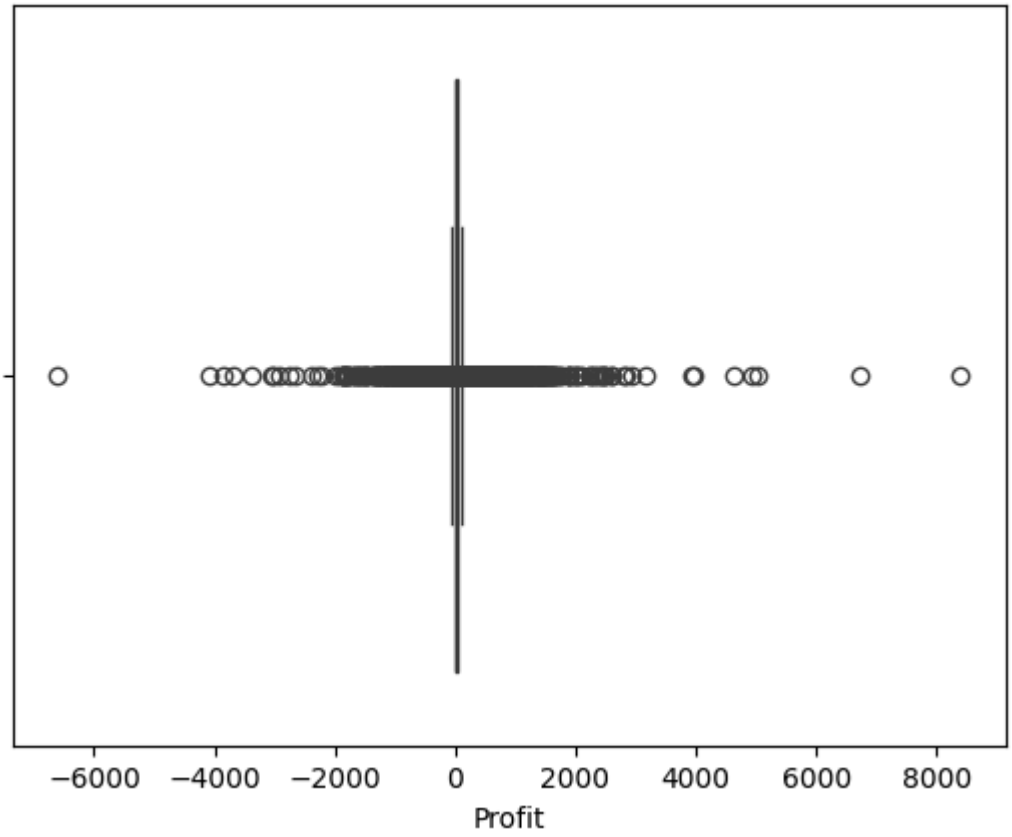
In [ ]:
```
for col in enumerate(num_new_data):
    sns.boxplot(x=col[1],data=num_new_data)
    plt.show()
```

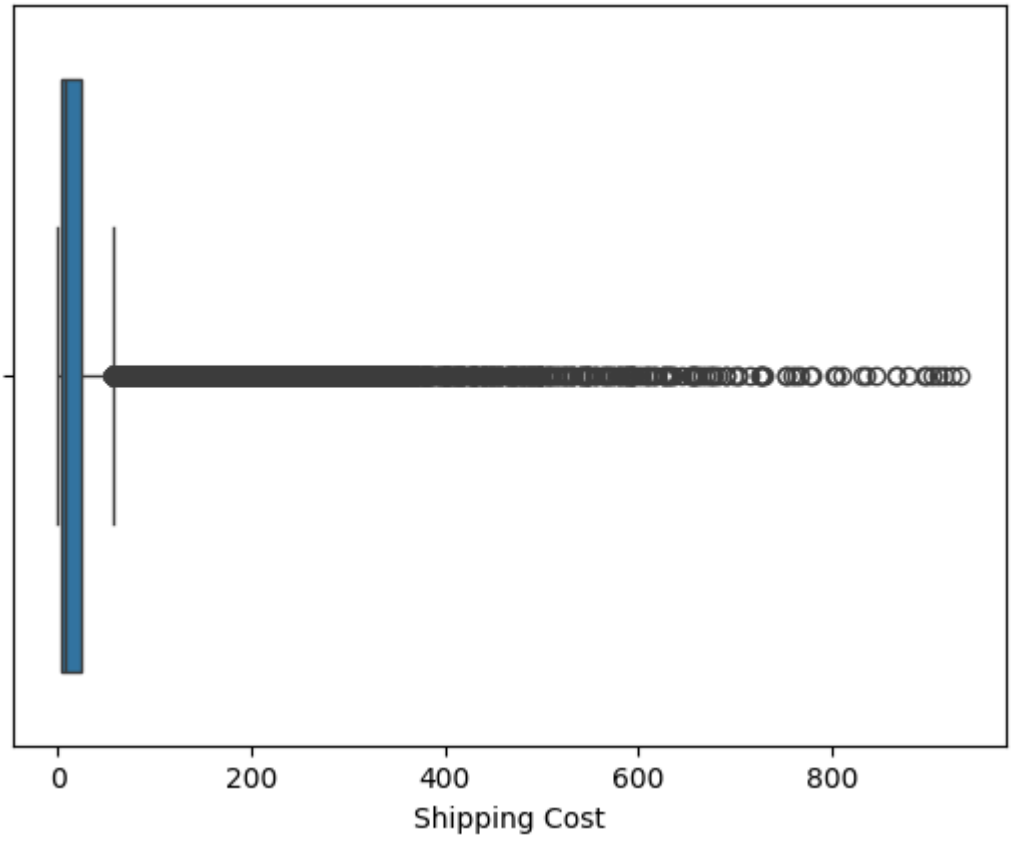

0
20000
40000
60000
80000
100000
Postal Code


0
5000
10000
15000
20000
Sales


2
4
6
8
10
12
14
Quantity


0.0
0.2
0.4
0.6
0.8
Discount





In [ ]:
```
#Using IQR
Q1 = num_new_data.quantile(0.25)

# obtain the third quartile
Q3 = num_new_data.quantile(0.75)

# obtain the IQR
IQR = Q3 - Q1
```

```
# print the IQR
print(IQR)
```

```
Postal Code      66785.000000
Sales              220.294575
Quantity             3.000000
Discount             0.200000
Profit              36.810000
Shipping Cost       21.840000
dtype: float64
```

In [ ]:
```
num_new_data.shape
```

Out[ ]:
```
(51290, 6)
```

In [ ]:
```
Q1 = num_new_data.quantile(0.25)
Q3 = num_new_data.quantile(0.75)
IQR = Q3 - Q1
```

In [ ]:
```
# Now perform the filtering
df_iqr = num_new_data[~((num_new_data < (Q1 - 1.5 * IQR)) | (num_new_data > (Q3
```